

11 — Fehlerbehandlung

Programme mit try / except / finally robust machen

HTW Berlin – Wirtschaftsingenieurwesen

SoSe 2026

Agenda

1. Wozu Fehlerbehandlung?
2. Was ist eine **Exception**?
3. try / except – Fehler abfangen
4. **Spezifische** Exception-Typen
5. Wichtige Exceptions im Überblick
6. Der finally-Block
7. Best Practices

Lernziel: Programme bauen, die nicht crashen, wenn der Anwender unsinnige Eingaben macht.

Wie wir heute arbeiten: Nach jedem Konzept zeigt die Folie *Live-Demo* (ich im Terminal) und *Sofort ausprobieren* (Sie im Notebook 11).

Wozu Fehlerbehandlung?

Ohne Behandlung

```
preis_text = input("Preis? ")
preis = float(preis_text)      # tippt "neunundachtzig"
                               # -> ValueError, Abbruch
```

Mit Behandlung

```
try:
    preis = float(input("Preis? "))
except ValueError:
    print("Bitte eine Zahl wie 89.95 eingeben.")
```

Robust = das Programm läuft auch dann sinnvoll weiter, wenn außen etwas schief geht.

Was ist eine Exception?

Traceback (most recent call last):

File "rechnung.py", line 5, in <module>

 summe = 100 / 0

ZeroDivisionError: division by zero

- Eine **Exception** (“Ausnahme”) ist Pythons Art, “etwas ist schiefgelaufen” zu signalisieren
- Das Traceback zeigt **wo** und **was**
- **Unbehandelt** → Programm endet sofort
- **Behandelt** → wir entscheiden, was passieren soll

try / except - die Grundstruktur

try:

Code, der eventuell schiefgehen könnte

```
bestellung = int(input("Stückzahl? "))
```

```
print(f"OK, {bestellung} Stück")
```

except:

Was tun, wenn ein Fehler auftritt?

```
print("Eingabe nicht verständlich.")
```

Block	Wann läuft er?
try:	immer (bis zum ersten Fehler)
except:	nur wenn im try eine Exception fliegt

try / except - Demo & Übung

► **Live-Demo** - 01_try_except_einfach.py

📝 **Sofort ausprobieren** - Notebook 11, Angeleitete Übung 1.1: Division mit Eingabe absichern

Spezifische Exception-Typen

```
try:
    pi      = float(input("Pi? "))
    radius  = float(input("Radius? "))
    umfang  = 2 * pi * radius
except ValueError:
    print("Bitte gültige Zahlen eingeben.")
except ZeroDivisionError:
    print("Division durch Null nicht erlaubt.")
```

- Jede except-Klausel **für einen Fehlertyp**
- Reihenfolge: spezifisch zuerst, allgemein zuletzt
- **Bare except:** (ohne Typ) fängt **alles** – nur sehr selten gewollt

Spezifische Typen - Demo & Übung

► **Live-Demo** - 02_spezifische_exceptions.py

✎ **Sofort ausprobieren** - Notebook 11, Angeleitete Übung 2.1: Kreisumfang mit ValueError

Wichtige Exception-Typen

Typ	Wann tritt er auf?	Beispiel
ValueError	richtiger Typ, falscher Wert	<code>int("abc")</code>
TypeError	falscher Typ für Operation	<code>"Hallo" + 5</code>
ZeroDivisionError	Teilen durch Null	<code>10 / 0</code>
KeyError	Schlüssel im Dict fehlt	<code>preise["Sandale"]</code>
IndexError	Listenindex zu groß	<code>liste[99]</code>
FileNotFoundError	Datei existiert nicht	<code>open("fehlt.txt")</code>

Fast jede Exception bekommt am Ende ein "Error" oder "Exception" - Klassennamen helfen beim Lesen des Tracebacks.

Mehrere Exceptions kompakt

```
try:
    eintrag = preise[modell]
    pro_stueck = gesamt / anzahl
except (KeyError, ZeroDivisionError):
    print("Prüfen Sie Modell und Stückzahl.")
```

- Mehrere Typen in einem except mit **Tupel**
- Praktisch, wenn die Reaktion für beide gleich ist

```
except ValueError as e:
    print(f"Fehler: {e}")
```

- as e – Exception-Objekt einfangen, um die Meldung anzuzeigen

Der finally-Block - immer aufräumen

```
try:
    summe = 100 / int(input("Teiler? "))
    print(summe)
except ZeroDivisionError:
    print("Null geht nicht.")
except ValueError:
    print("Keine Zahl.")
finally:
    print("Prüfung abgeschlossen.")
```

- finally läuft **immer** – mit oder ohne Fehler, mit oder ohne except-Treffer
- Typisch für **Aufräumen**: Datei schließen, Verbindung trennen, Statusmeldung

► **Live-Demo** - 03_finally_block.py

✎ **Sofort ausprobieren** - Notebook 11, Angeleitete Übung 3.1:
try/except/finally mit Statusmeldung

Schleife + try/except - robuste Eingabe

```
while True:
    try:
        anzahl = int(input("Wie viele Eco-Sneaker? "))
        break
    except ValueError:
        print("Bitte eine ganze Zahl, z. B. 3.")
print(f"OK, {anzahl} Stück.")
```

- while True + break aus Notebook 10
- Bei jeder **falschen** Eingabe nur Hinweis + erneut fragen
- Bei der **ersten** korrekten Eingabe verlässt break die Schleife

► **Live-Demo** - 04_eingabe_validieren.py

Best Practices - und was zu vermeiden ist

schlecht:

try:

...

except:

schluckt alles, sogar Strg+C

pass

und tut dann nichts

gut:

try:

...

except (ValueError, TypeError) **as** e:

print(f"Eingabe-Fehler: {e}")

Best Practices - Faustregeln

Tu	Lass
Spezifische Exceptions abfangen	except: ohne Typ
Hilfreiche Meldung ausgeben	stillschweigend ignorieren
Nur den Code im try halten, der wirklich fehlerhaft sein kann	ganze Funktionen umhüllen

Cheat Card

Aufgabe	Syntax
Fehler abfangen	<code>try: ... except Type: ...</code>
Mehrere Typen	<code>except (T1, T2): ...</code>
Meldung lesen	<code>except T as e: print(e)</code>
Immer ausführen	<code>finally: ...</code>
Eingabe absichern	<code>while True: try ... except ...</code>

Faustregel: Fehlerbehandlung ist **für den Anwender da**, nicht zum Verstecken eigener Bugs. Fangen Sie nur, wovon Sie wissen, was Sie tun.

Dateien sind ein **Hotspot** für Fehler – es gibt Pfade, Berechtigungen, fehlende Dateien:

```
try:
    with open("preise.csv") as f:
        text = f.read()
except FileNotFoundError:
    print("preise.csv fehlt -- Standardwerte werden genutzt.")
```

try/except plus with ist das **Standard-Muster** beim Lesen und Schreiben von Dateien.

- ✓ Was eine Exception ist und wie ein Traceback aussieht
- ✓ try / except als Grundstruktur
- ✓ Spezifische Exception-Typen unterscheiden
- ✓ Mehrere except-Blöcke kombinieren
- ✓ finally für garantierte Aufräumarbeit
- ✓ Robuste Eingabe-Schleife mit while True

Zur Vertiefung im Notebook 11:

- Angeleitete Übungen 1.1, 1.2, 2.1, 3.1 (sind Sie schon mitgegangen)
- Abschlussübungen Teil 1 (Grundlagen) und Teil 2 (Transfer)
- Musterlösungen am Notebook-Ende zur Selbstkontrolle